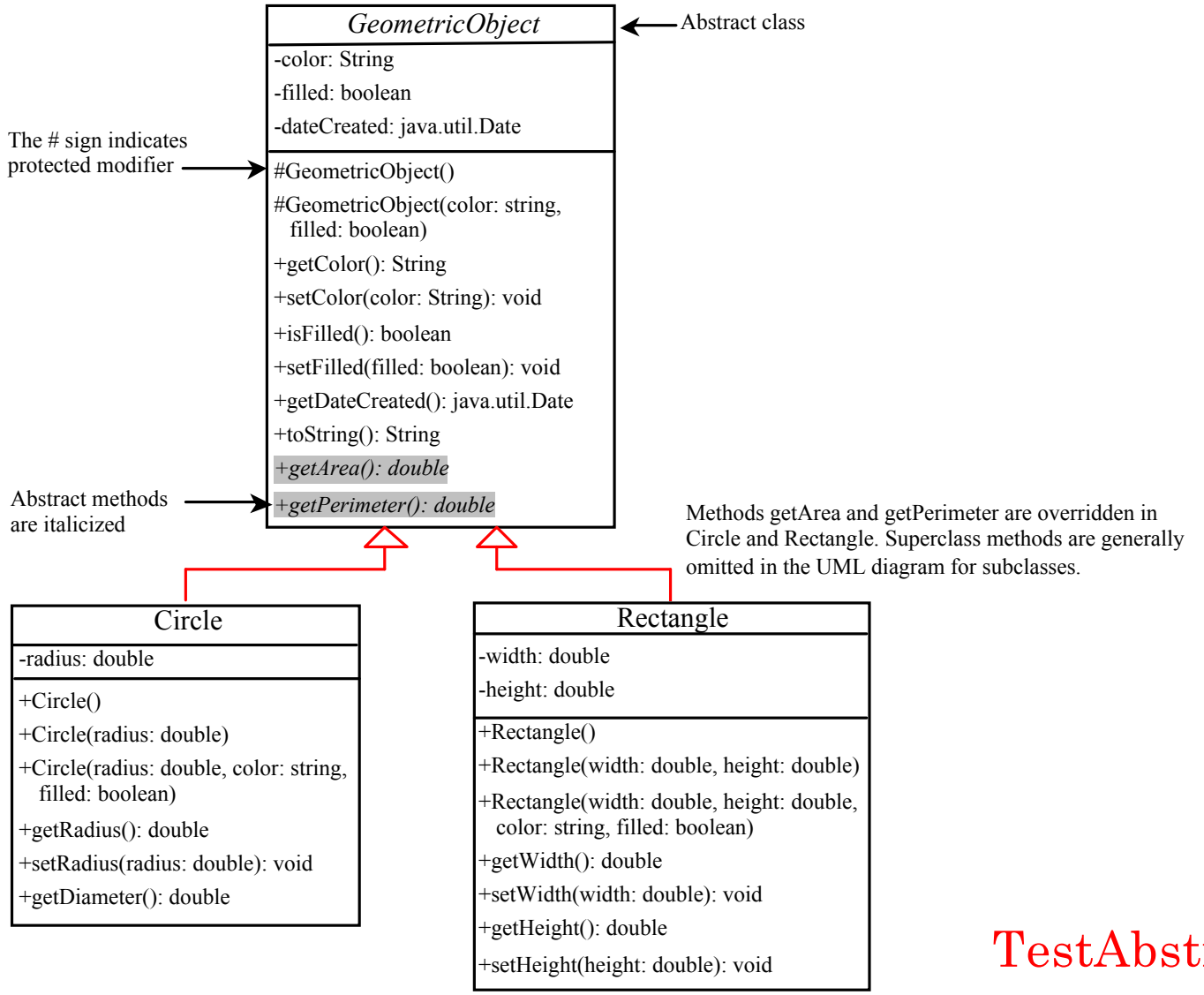
Object Oriented Programming

Mihai Dascălu

Abstract classes

Abstract Classes and Abstract Methods



TestAbstractGO

Abstract Method in Abstract Class

- An abstract method cannot be contained in a non-abstract class
- If a subclass of an abstract superclass **does not implement all** the **abstract methods**
  => The subclass must be defined as **abstract**
- In a non-abstract subclass extended from an abstract class
  => All **abstract methods must be implemented,** even if they are not used in the subclass

Objects cannot be created from abstract classes

- An abstract class cannot be instantiated using the new operator, but you can still **define its constructors**
  - These are invoked in the constructors of its subclasses
- For example:
  - The constructors of GeometricObject are invoked in the Circle class and the Rectangle class

Abstract class without abstract method

- It is possible to **define** an **abstract** class that contains **no abstract methods**
- In this case, you cannot create instances of the class using the new operator
  - ⇒this class is used as a base class for defining a new subclass
  - ⇒cheating: { };

Superclass of abstract class may be concrete

- A subclass can be abstract even if its superclass is concrete
- For example:
  - The Object class is concrete, but its subclasses, such as GeometricObject, may be abstract

Concrete method overridden to be abstract

- A subclass can override a method from its superclass to define it abstract
- This is rare, but useful when the implementation of the method in the superclass becomes invalid in the subclass

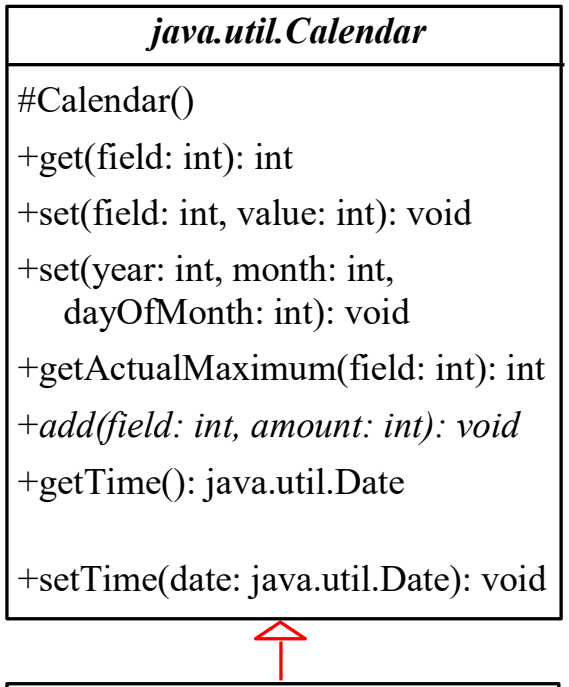
  => The *subclass must be defined as abstract*

Abstract Class as Type

- You cannot create an instance from an abstract class using the new operator
- However, an abstract class can be used as a data type:

```
GeometricObject[] geo = new GeometricObject[10];
```

The Abstract Calendar Class and Its GregorianCalendar subclass



Constructs a default calendar.

Returns the value of the given calendar field.

Sets the given calendar to the specified value.

Sets the calendar with the specified year, month, and date. The month parameter is 0-based, that is, 0 is for January.

Returns the maximum value that the specified calendar field could have.

Adds or subtracts the specified amount of time to the given calendar field.

Returns a Date object representing this calendar's time value (million second offset from the Unix epoch).

Sets this calendar's time with the given Date object.

Constructs a GregorianCalendar for the current time.

Constructs a GregorianCalendar for the specified year, month, and day of month.

Constructs a GregorianCalendar for the specified year, month, day of month, hour, minute, and second. The month parameter is 0-based, that is, 0 is for January.

The Abstract Calendar Class and its GregorianCalendar Subclass

- An instance of java.util.Date represents a specific instant in time with millisecond precision
- java.util.Calendar = *abstract base class* for extracting detailed information such as year, month, date, hour, minute and second from a Date object
- Subclasses of Calendar can implement specific calendar systems such as Gregorian calendar (java.util.GregorianCalendar), Lunar Calendar and Jewish calendar

The GregorianCalendar Class

- new GregorianCalendar() => construct a default GregorianCalendar with the current time
- new GregorianCalendar(year, month, date) => construct a GregorianCalendar with the specified year, month, and date
- The month parameter is 0-based, i.e., 0 = January

The get Method in Calendar Class

- get(int field) method from Calendar class = extracts the date and time information from a Calendar object
- fields are defined as constants

Constant	Description
YEAR	The year of the calendar.
MONTH	The month of the calendar with 0 for January.
DATE	The day of the calendar.
HOUR	The hour of the calendar (12-hour notation).
HOUR_OF_DAY	The hour of the calendar (24-hour notation).
MINUTE	The minute of the calendar.
SECOND	The second of the calendar.
DAY_OF_WEEK	The day number within the week with 1 for Sunday.
DAY_OF_MONTH	Same as DATE.
DAY_OF_YEAR	The day number in the year with 1 for the first day of the year.
WEEK_OF_MONTH	The week number within the month.
WEEK_OF_YEAR	The week number within the year.
AM_PM	Indicator for AM or PM (0 for AM and 1 for PM).

TestCalendar

Interfaces

What is an interface? Why is an interface useful?

- A class-like construct that *contains only constants and abstract methods*
- An interface resembles an abstract class, but the **intent** of an interface is to specify **behavior** for objects
  - e.g., specify that the objects are comparable or cloneable using appropriate interfaces
- An **interface** can extend any number of **interfaces**
  - An **interface** cannot **implement another interface**
    - **A:** if any **interface** is implemented, then its methods must be defined
    - However, **interfaces** do not include method definitions*
- Default & static methods in interfaces since JDK 8+
- Private methods in interfaces since JDK 9+

Edible

Define an Interface

- To distinguish an interface from a class, Java uses the following syntax to define an interface:

```
public interface InterfaceName {
  constant declarations;
  method signatures;
}
```

- Example

```
public interface Edible {
  /** Describe how to eat */
  public abstract String howToEat();
}
```

Interface is a Special Class

- An interface is treated like a special class in Java
- Each interface is compiled into a separate bytecode file, just like a regular class
- Like an abstract class, you cannot create an instance from an interface using the new operator
- In most cases, you can use an interface in the same way as an abstract class
  - For example, you can use an interface as:
    - A data type for a variable
    - The result of casting

Example

- You can now use the Edible interface to specify whether an object is edible
- This is accomplished by letting the class for the object implement this interface using the implements keyword
- For example, the classes Chicken and Fruit implement the Edible interface

TestEdible
PressSwitch

Omitting Modifiers in Interfaces

- All data fields are public final static
- All method declarations in an interface, including static methods, are implicitly public (except private methods)
  => these *modifiers can be omitted*

```
public interface T1 {
  public static final int K = 1;

  public abstract void p();
}
```

Equivalent

```
public interface T1 {
  int K = 1;

  void p();
}
```

- A constant defined in an interface can be accessed using syntax InterfaceName.CONSTANT_NAME (e.g., T1.K)
  - Similar for static methods

What can you have in an interface?!?

- Constant variables
- Abstract methods
- Implementable:
  - Default methods
  - Static methods
  - Private methods
  - Private Static methods

Bring me the new stuff

- Java 1.8: **default methods**
  - Allow the interfaces to have methods with implementation, without affecting the classes that implement the interface
- Java 9: **private methods**
  - No need to write duplicate code, hence more code reusability
  - Introduce the choice to expose only the intended methods implementations
  - Private methods should have full implementation
  - Private methods **must** contain a body

Razboinic

Rules concerning private methods in interfaces

- Private interface methods cannot be abstract
- Private methods can be used only inside the interface
- Private static methods can be used inside other static and non-static interface methods
- Private non-static methods cannot be used inside private static methods

The Comparable Interface

```
// This interface is defined in
// java.lang package
package java.lang;


public interface Comparable {
  public int compareTo(Object o);
}
```

String and Date Classes

- Many classes (e.g., String and Date) in the Java library implement Comparable to define a natural order for the objects
- If you examine the source code of these classes, you will see the keyword implements used in the classes:

```
public class String extends Object
    implements Comparable {
  // class body omitted
}
```

```
public class Date extends Object
    implements Comparable {
  // class body omitted
}
```

```
new String() instanceof String
new String() instanceof Comparable
new java.util.Date() instanceof java.util.Date
new java.util.Date() instanceof Comparable
```

Generic `max` Method

- The return value from the max method is of the Comparable type. So, you need to cast it to String or Date explicitly

```
String s1 = "abcdef";
String s2 = "abcdee";
String s3 = (String)Max.max(s1, s2);
```

```
Date d1 = new Date();
Date d2 = new Date();
Date d3 = (Date)Max.max(d1, d2);
```

```
// Max.java: Find a maximum object
public class Max {
  /** Return the maximum of two objects */
  public static Comparable max
      (Comparable o1, Comparable o2) {
    if (o1.compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

(a)

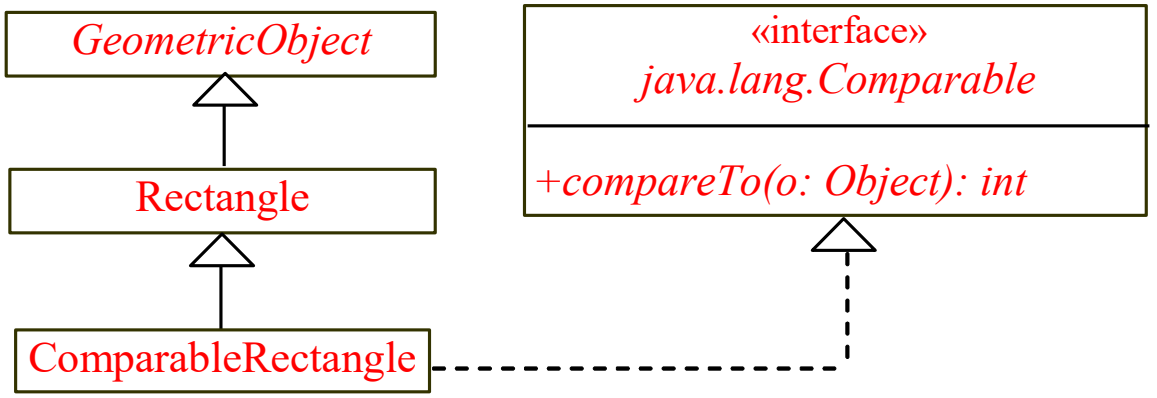
```
// Max.java: Find a maximum object
public class Max {
  /** Return the maximum of two objects */
  public static Object max
      (Object o1, Object o2) {
    if (((Comparable)o1).compareTo(o2) > 0)
      return o1;
    else
      return o2;
  }
}
```

(b)

Defining Classes to Implement Comparable

- max method cannot be applied on two instances of Rectangle that do not implement Comparable =>
- Define a new rectangle class that implements Comparable => instances of this new class are comparable

Notation:
The interface name and the method names are italicized. The dashed lines and hollow triangles are used to point to the interface.




```
ComparableRectangle rectangle1 = new ComparableRectangle(4, 5);
ComparableRectangle rectangle2 = new ComparableRectangle(3, 6);
System.out.println(Max.max(rectangle1, rectangle2));
```

ComparableRectangle

The `Cloneable` Interfaces

- **Marker** Interface = An empty interface
- A marker interface does not contain constants or methods

  => Denote that a class possesses certain desirable properties
- A class that implements the Cloneable interface is marked cloneable, and its objects can be cloned using the clone() method defined in the Object class

```
package java.lang;

public interface Cloneable {
}
```

Examples

- Many classes (e.g., Date and Calendar) in the Java library implement Cloneable => instances can be cloned

```
Calendar calendar = new GregorianCalendar(2003, 2, 1);

Calendar calendarCopy = (Calendar)calendar.clone();

System.out.println("calendar == calendarCopy is "
+(calendar == calendarCopy));

System.out.println("calendar.equals(calendarCopy) is "
+ calendar.equals(calendarCopy));
```

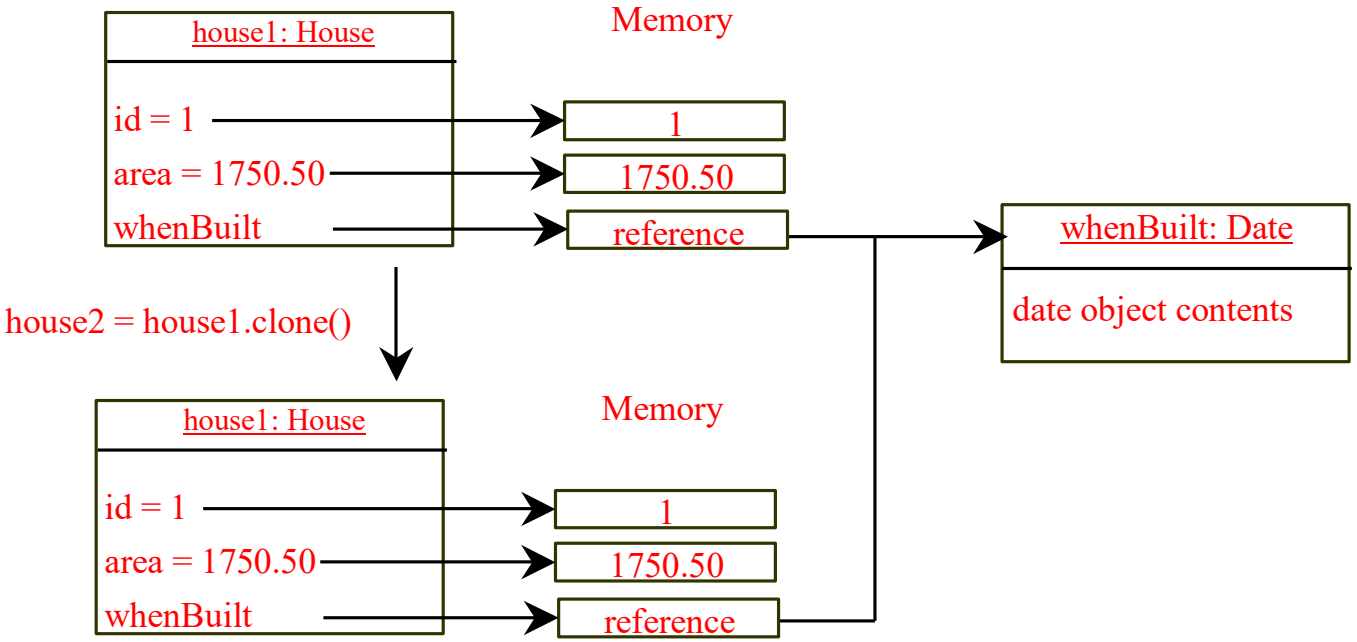
- Displays

```
calendar == calendarCopy is false

calendar.equals(calendarCopy) is true
```

Implementing Cloneable Interface (Shallow versus Deep Copy)

- To define a custom class that implements the Cloneable interface, the class must override the clone() method in the Object class
- `House house1 = new House(1, 1750.50);`
- `House house2 = (House)house1.clone();`



House

Interfaces versus Abstract Classes (1)

- Data must contain only constants in an interface
- An abstract class can have all types of data

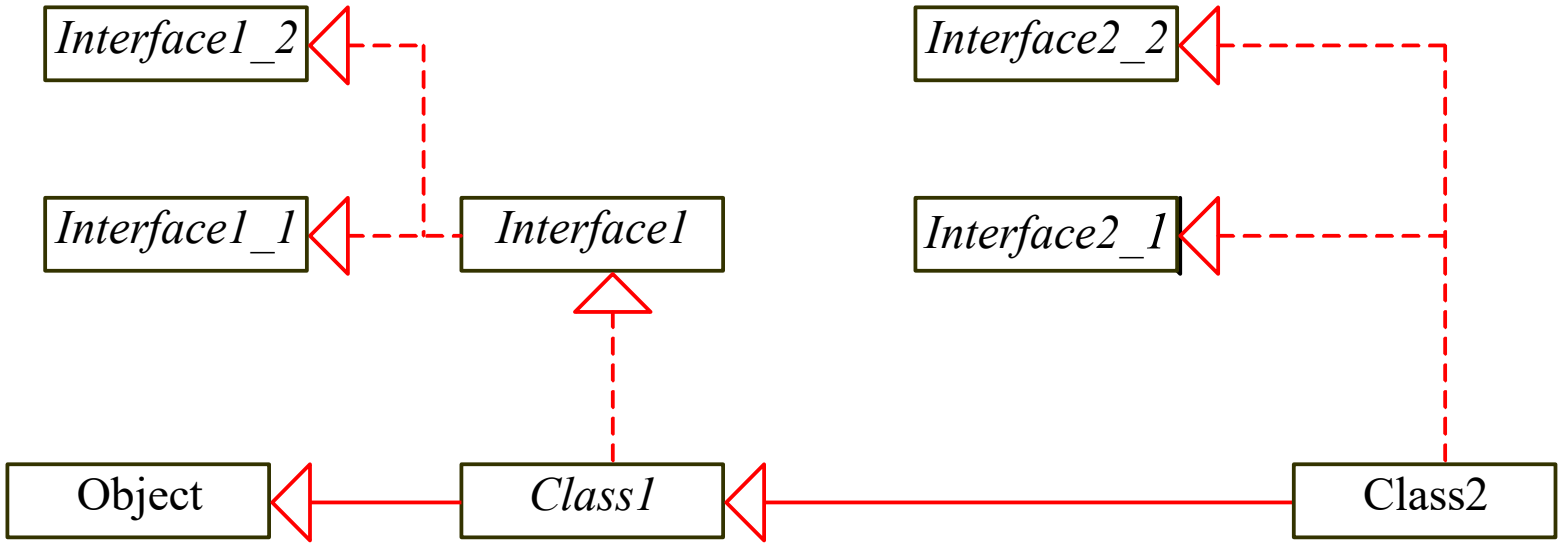
- Each method in an interface has only a signature without implementation
- An abstract class can have concrete methods

	Variables	Constructors	Methods
Abstract class	No restrictions	Constructors are invoked by subclasses through constructor chaining An abstract class cannot be instantiated using *new*	No restrictions
Interface	All variables must be public static final	No constructors An interface cannot be instantiated using *new*	All methods must be public abstract instance methods

Interfaces versus Abstract Classes (2)

- All classes share a single root, the Object class, but there is no single root for interfaces
- Like a class, an interface also defines a type
- A variable of an interface type can reference any instance of the class that implements the interface
- **Interfaces** can **extend 1+ interfaces** that play the same role as a superclass
- You can use an interface as a data type and cast a variable of an interface type to its subclass, and vice versa

Interfaces versus Abstract Classes (3)



- Suppose that *c* is an instance of Class2
- *c* is also an instance of Object, Class1, Interface1, Interface1_1, Interface1_2, Interface2_1, and Interface2_2.

Caution: conflict interfaces

- In rare occasions, a class may implement two interfaces with conflict information
  - Two same constants with different values
  - Two methods with same signature, but different return types
- This type of errors will be detected by the compiler

Interface or an Abstract Class?

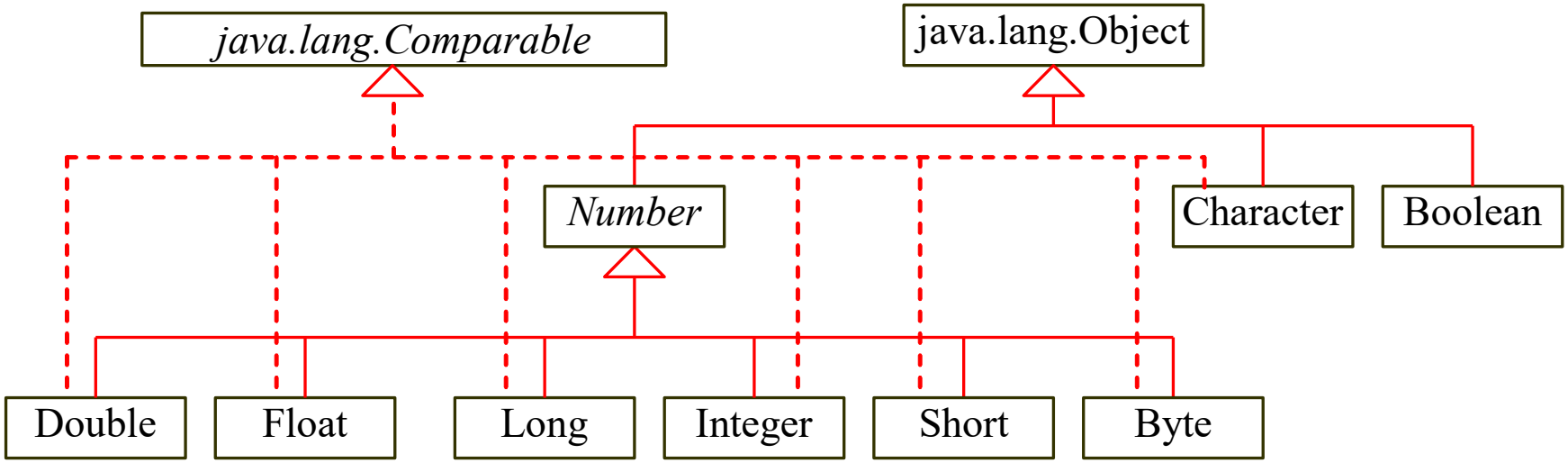
- Abstract classes and interfaces can both be used to model common features
- A **strong is-a relationship** that clearly describes a parent-child relationship should be modeled using abstract **classes**
- A **weak is-a relationship**, also known as an **is-kind-of** relationship, indicates that an object possesses a certain property => **interfaces**
- In case of **multiple inheritance**, you have to design **one** as a **superclass**, and **others** as **interface**

Wrapper Classes

- Boolean
- Character
- Short
- Byte
- Integer
- Long
- Float
- Double

NOTE: (1) The wrapper classes do not have no-arg constructors

(2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created



The toString, equals, and hashCode Methods

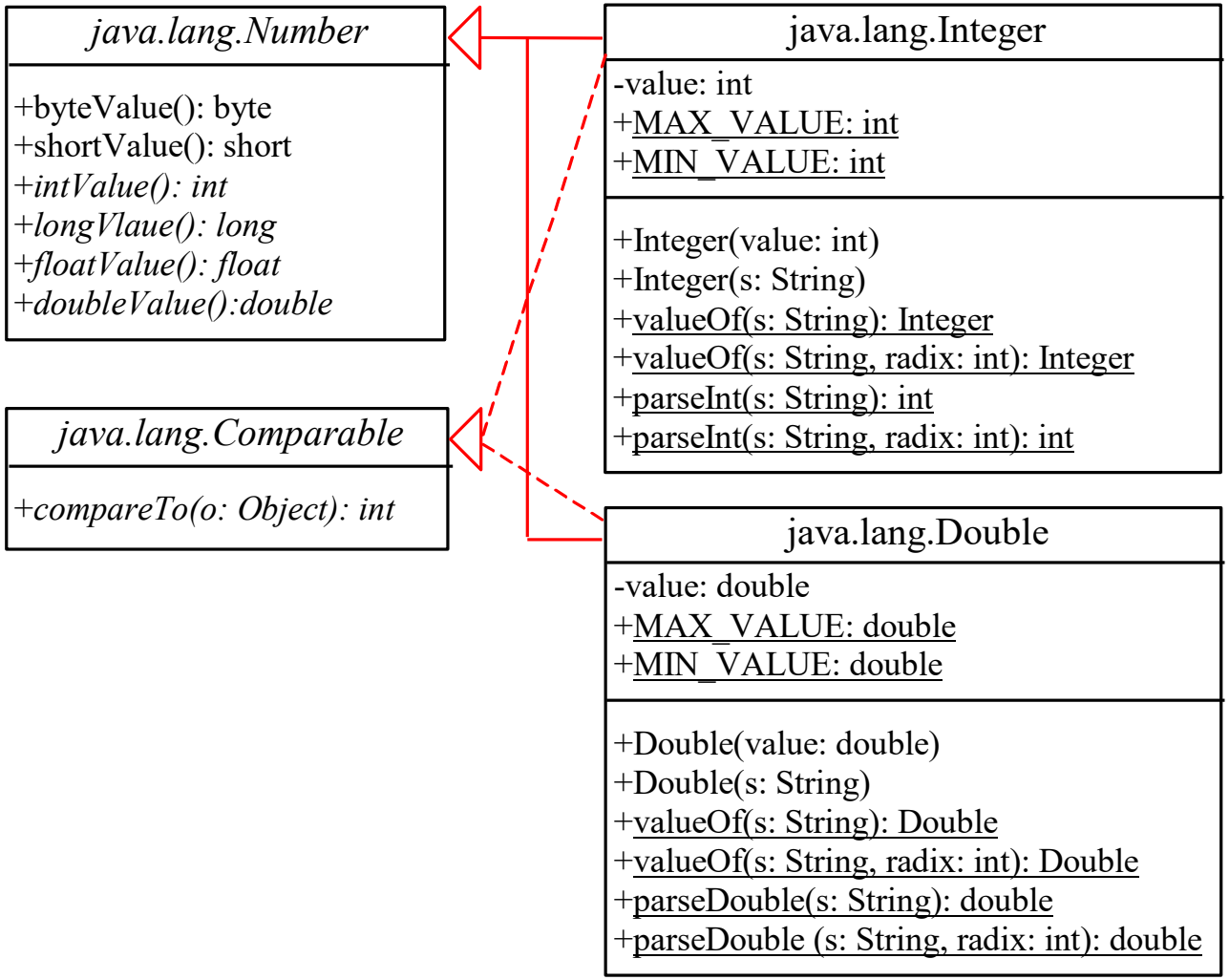
- Each wrapper class overrides the toString, equals, and hashCode methods defined in the Object class
- Since all the numeric wrapper classes and the Character class implement the Comparable interface

  => the compareTo method is implemented in these classes

The Number Class

- Each numeric wrapper class extends the **abstract Number class**, which contains the methods:
  - doubleValue, floatValue, intValue, longValue, shortValue, and byteValue
- These methods "convert" objects into primitive type values
- The methods doubleValue, floatValue, intValue, longValue are abstract
- The methods byteValue and shortValue are not abstract, which simply return (byte)intValue() & (short)intValue()

The Integer and Double Classes



Numeric Wrapper Class Constructors

- Internal methods
  - Class Constants MAX_VALUE, MIN_VALUE
  - Conversion Methods
- You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value.
- The constructors for Integer and Double are:
  - public Integer(int value) **- Deprecated**
  - public Integer(String s) **- Deprecated**
  - public Double(double value) **- Deprecated**
  - public Double(String s) **- Deprecated**

Numeric Wrapper Class Constants

- Each numerical wrapper class has the constants MAX_VALUE and MIN_VALUE. MAX_VALUE:
  - the maximum value of the corresponding primitive data type
- MIN_VALUE represents the minimum byte, short, int, and long values for Byte, Short, Integer, and Long
- MIN_VALUE represents the minimum *positive* float and double values for Float and Double,

Conversion Methods

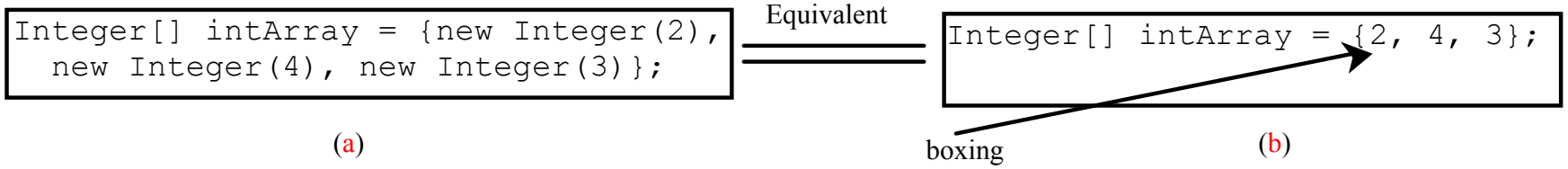
- Each numeric wrapper class implements the abstract methods doubleValue, floatValue, intValue, longValue, and shortValue, defined in the Number class
- These methods "convert" objects into primitive type values
- The numeric wrapper classes have a useful class method, valueOf(String s) => create a new object initialized to the value represented by the specified String:
  - Double doubleObject = Double.valueOf("12.4");
  - Integer integerObject = Integer.valueOf("12");

CAUTION

- Although an int value can be assigned to a double type variable, int[] and double[] are two incompatible types
- You cannot assign an int[] array to a variable of double[] or Object[] type

Automatic Conversion Between Primitive Types and Wrapper Class Types

- Primitive types and wrapper classes can be converted automatically:




```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Unboxing

```
System.out.println(intArray[0] + (intArray[1] + intArray[2]));
```

BigInteger and BigDecimal

- Need to use very large integers or high precision floating-point values,
  - BigInteger & BigDecimal classes in the java.math package
- Both are *immutable*
- Both extend the Number class and implement the Comparable interface